

ZERO-RUPEE TERMUX WEBSITE HOSTING

Host a real public website from an Android phone using mobile data, IPv6, DuckDNS and Caddy

Complete copy-paste guide — installation, DNS, HTTPS, automatic IPv6 updates, one-command startup, reboot automation, example premium website, troubleshooting and security.

Component	Role
Android + Termux	The actual web server
Airtel/other mobile IPv6	Direct Internet connectivity
DuckDNS	Free hostname + IPv6 DNS updates
Caddy + DuckDNS plugin	HTTPS / Let's Encrypt via DNS-01
Termux:Boot	Start the server after reboot

What this guide produces

`https://YOURNAME.duckdns.org:8443` served directly by your Android phone.

This guide intentionally uses an unprivileged HTTPS port (8443). Standard ports 80/443 cannot normally be bound by an unrooted Termux process. DNS-01 avoids the need for port 80/443 during certificate issuance.

1. What this setup can and cannot do

The setup below is a genuine home/mobile server: the website files live on the Android device, Caddy serves them, and the phone's IPv6 address is published in DNS. No paid VPS or hosting account is required.

Important IPv6 limitation: if the mobile carrier gives you only CGNAT IPv4 plus IPv6, the direct server is normally reachable only by clients that have working IPv6 connectivity. IPv4-only visitors cannot directly reach an IPv6-only server. A tunnel or dual-stack/VPS arrangement is required for broad IPv4 reachability.

A public IPv4 shown by a site is not enough. If Android itself has an address in 100.64.0.0/10, that is carrier-grade NAT space. Direct inbound IPv4 is not available through that address.

Prerequisites

- Android phone with Termux and working mobile data.
- A globally routable IPv6 address on the mobile interface.
- A free DuckDNS account and a DuckDNS hostname.
- Enough battery/power for an always-on server; Android may otherwise kill background apps.
- For automatic startup after reboot: the Termux:Boot add-on installed from the same trusted distribution source as Termux.

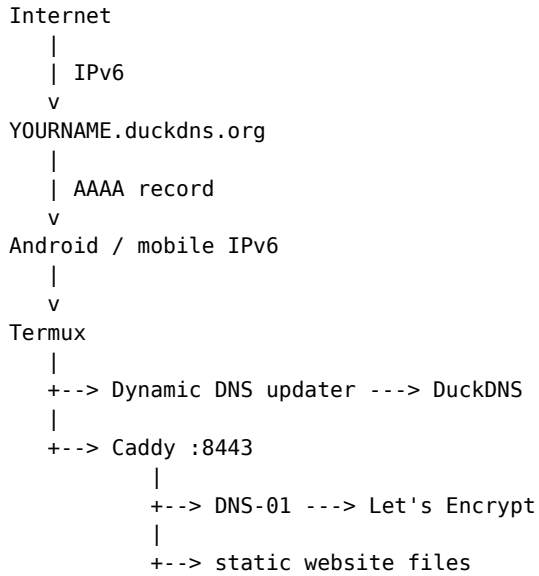
Install Termux

Use a current, trusted Termux distribution. Termux:Boot documentation states that the add-on is intended to execute scripts at boot and that it should be started once after installation before using `~/.termux/boot/`.

Official references:

<https://termux.dev/> <https://github.com/termux/termux-boot>

2. Final architecture



Traffic to the site uses HTTPS on TCP 8443. Caddy obtains/renews the certificate using the DNS-01 challenge, which does not require the certificate authority to connect to your server. Caddy's documentation explicitly notes that DNS-01 requires no open ports for validation.

The DuckDNS Caddy module provides the `dns.providers.duckdns` module and supports the Caddyfile form `tls { dns duckdns {env.DUCKDNS_API_TOKEN} }`.

References: [Caddy Automatic HTTPS](#) and [DuckDNS module documentation](#).

3. Stage 1 — test your Android as a web server

Create a simple test page first. This isolates networking problems from Caddy/DNS problems.

```
pkg update
pkg install python curl iproute2
mkdir -p ~/testserver
cd ~/testserver
echo '<h1>Hello from my Android!</h1>' > index.html
python -m http.server 8080 --bind ::
```

Find the global IPv6:

```
ip -6 addr show rmnet0
```

If your mobile interface is not rmnet0, find the interface carrying the default IPv6 route:

```
ip -6 route show default
```

Then test locally with brackets around IPv6:

```
curl -6 "http://[YOUR_IPV6]:8080"
```

If you get your HTML response, the local web server works. External reachability must be tested from a different network or an independent IPv6 port tester.

4. Stage 2 — create a free DuckDNS hostname

Create a hostname such as `yourname.duckdns.org` at DuckDNS. Keep your DuckDNS token private; treat it like a credential.

Official site: <https://www.duckdns.org/>

After creating the hostname, the update API can publish your current IPv6 using the `ipv6=` parameter.

Create a secure token file:

```
mkdir -p ~/duckdns
nano ~/.duckdns_token

# Paste ONLY your DuckDNS token into the file.
# Save: CTRL+O, ENTER
# Exit: CTRL+X

chmod 600 ~/.duckdns_token
```

Do not put the token in a file that you plan to publish to GitHub. Do not paste it into screenshots, tutorials, chat messages, or public repositories.

5. Stage 3 — automatic IPv6 → DuckDNS updater

The following version is more portable than hard-coding `rmnet0`: it asks Linux which source IPv6 would be used for an external IPv6 route, then publishes that address.

Create the updater:

```
nano ~/duckdns/update-ipv6.sh
```

Paste:

```
#!/data/data/com.termux/files/usr/bin/bash

DOMAIN="YOUR_DUCKDNS_SUBDOMAIN"
TOKEN_FILE="$HOME/.duckdns_token"

if [ ! -r "$TOKEN_FILE" ]; then
    echo "ERROR: $TOKEN_FILE not found."
    exit 1
fi

TOKEN="$(cat "$TOKEN_FILE")"

# Find the IPv6 address used for the default IPv6 route.
IPV6="$(
    ip -6 route get 2606:4700:4700::1111 2>/dev/null |
    awk '{
        for (i=1; i<=NF; i++)
            if ($i == "src") { print $(i+1); exit }
    }'
)"

if [ -z "$IPV6" ]; then
    echo "ERROR: No usable global IPv6 was found."
    exit 1
fi

RESULT="$(
    curl -fsS \
        --get \
        --data-urlencode "domains=$DOMAIN" \
        --data-urlencode "token=$TOKEN" \
        --data-urlencode "ipv6=$IPV6" \
        "https://www.duckdns.org/update"
)"

if [ "$RESULT" = "OK" ]; then
    echo "DuckDNS updated: $DOMAIN -> $IPV6"
else
    echo "DuckDNS update failed: $RESULT"
    exit 1
fi
```

Replace `YOUR_DUCKDNS_SUBDOMAIN` with only the subdomain part, e.g. `abdullahferozonline`.

```
chmod +x ~/duckdns/update-ipv6.sh
~/duckdns/update-ipv6.sh
```

Expected result:

```
DuckDNS updated: YOUR_DUCKDNS_SUBDOMAIN -> 2401:....
```

Verify DNS:

```
pkg install dnsutils
dig AAAA YOUR_DUCKDNS_SUBDOMAIN.duckdns.org
```

6. Stage 4 — install/build Caddy with DuckDNS DNS-01 support

The standard Caddy package does not necessarily contain the DuckDNS DNS provider module. Build a custom Caddy binary with the community DuckDNS module.

```
pkg update
pkg install golang git clang
go install github.com/caddyserver/xcaddy/cmd/xcaddy@latest
export PATH="$PATH:$(go env GOPATH)/bin"
export CGO_ENABLED=1

cd ~/testserver/YOUR_WEBSITE_DIRECTORY
xcaddy build --with github.com/caddy-dns/duckdns
```

Verify the custom binary:

```
./caddy version
./caddy list-modules | grep dns.providers.duckdns
```

The second command must print `dns.providers.duckdns`. The DuckDNS module documentation identifies that exact module name.

Why `CGO_ENABLED=1`? On some Android/ARM builds, `xcaddy/Go` needs external C linking. Installing `clang` and enabling `CGO` avoids the build failure encountered on some Termux ARM environments.

7. Stage 5 — Caddy HTTPS configuration

Create the configuration:

```
mkdir -p ~/caddy
nano ~/caddy/Caddyfile
```

Paste:

```
{
    https_port 8443
    auto_https disable_redirects
}

YOUR_DUCKDNS_SUBDOMAIN.duckdns.org:8443 {
    tls {
        dns duckdns {env.DUCKDNS_API_TOKEN}
    }

    root * /data/data/com.termux/files/home/testserver/YOUR_WEBSITE_DIRECTORY
    file_server
}
```

Replace the hostname and website directory. Then:

```
./caddy fmt --overwrite ~/caddy/Caddyfile
./caddy validate --config ~/caddy/Caddyfile
```

Set the token only in the process environment:

```
export DUCKDNS_API_TOKEN="$(cat ~/.duckdns_token)"
./caddy run --config ~/caddy/Caddyfile
```

Why DNS-01? Caddy's normal HTTP-01 challenge needs port 80, while TLS-ALPN-01 needs port 443. DNS-01 instead creates a temporary DNS TXT record and therefore avoids the privileged-port problem.

□cite-style-note□

8. Stage 6 — one-command startup

This launcher updates DuckDNS immediately, keeps updating it every 5 minutes, exports the Caddy DNS credential, prevents sleep when the Termux API is available, validates Caddy, and starts the server.

```
nano ~/start-site
```

Paste:

```
#!/data/data/com.termux/files/usr/bin/bash
set -u

DOMAIN="YOUR_DUCKDNS_SUBDOMAIN"
TOKEN_FILE="$HOME/.duckdns_token"
CADDY="$HOME/testserver/YOUR_WEBSITE_DIRECTORY/caddy"
CADDYFILE="$HOME/caddy/Caddyfile"

if [ ! -r "$TOKEN_FILE" ]; then
    echo "ERROR: $TOKEN_FILE is missing."
    exit 1
fi

if [ ! -x "$CADDY" ]; then
    echo "ERROR: Custom Caddy binary not found:"
    echo "$CADDY"
    exit 1
fi

TOKEN="$(cat "$TOKEN_FILE")"

update_dns() {
    IPV6="$(
        ip -6 route get 2606:4700:4700::1111 2>/dev/null |
        awk '{
            for (i=1; i<=NF; i++)
                if ($i == "src") { print $(i+1); exit }
        }'
    )"

    if [ -z "$IPV6" ]; then
        echo "[$(date)] No usable global IPv6."
        return 1
    fi

    RESULT="$(
        curl -fsS --get \
            --data-urlencode "domains=$DOMAIN" \
            --data-urlencode "token=$TOKEN" \
            --data-urlencode "ipv6=$IPV6" \
            "https://www.duckdns.org/update"
    )"

    if [ "$RESULT" = "OK" ]; then
        echo "[$(date)] DuckDNS OK: $IPV6"
    else
        echo "[$(date)] DuckDNS FAILED: $RESULT"
    fi
}

cleanup() {
    [ -n "${DNS_PID:-}" ] && kill "$DNS_PID" 2>/dev/null || true
    command -v termux-wake-unlock >/dev/null 2>&1 && termux-wake-unlock || true
}

trap cleanup EXIT INT TERM

echo "=== Termux Website Server ==="

command -v termux-wake-lock >/dev/null 2>&1 && termux-wake-lock

update_dns

export DUCKDNS_API_TOKEN="$TOKEN"
```

```
"$CADDY" validate --config "$CADDYFILE" || exit 1

(
  while true; do
    sleep 300
    update_dns
  done
) &
DNS_PID=$!

echo
echo "LIVE:"
echo "https://$DOMAIN.duckdns.org:8443"
echo

exec "$CADDY" run --config "$CADDYFILE"
```

Replace the two placeholders, then:

```
chmod +x ~/start-site
~/start-site
```

Your one-command operation is now:

```
~/start-site
```

Keep this terminal/session alive. Android battery optimization can still kill background work; the boot section below improves persistence.

9. Stage 7 — automatic startup after reboot

Termux:Boot runs scripts placed in `~/.termux/boot/`. Its documentation also recommends using `termux-wake-lock` if you need the device kept awake.

Install and open Termux:Boot once, then create the boot directory:

```
mkdir -p ~/.termux/boot
nano ~/.termux/boot/start-site
```

Paste:

```
#!/data/data/com.termux/files/usr/bin/bash

termux-wake-lock
sleep 10

# Optional: wait for mobile networking to come up.
for i in $(seq 1 30); do
    if ip -6 route show default | grep -q '^default'; then
        break
    fi
    sleep 2
done

exec "$HOME/start-site"

chmod +x ~/.termux/boot/start-site
```

Reboot test:

After reboot, wait for mobile data/IPv6 to become available. Termux:Boot should launch the script. If Android aggressively stops Termux, remove battery optimization for Termux and Termux:Boot in Android settings where your device exposes that option.

The Termux:Boot project explicitly says to install the add-on, launch it once, create `~/.termux/boot/`, and place executable scripts there.

10. Premium example website

Create a folder and add three files. This example is framework-free and therefore ideal for a small Termux server.

```
cd ~/testserver
mkdir -p premium-site
cd premium-site
nano index.html
```

index.html:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <meta name="description" content="A premium mobile-first website hosted from Termux.">
  <title>Nova – Premium Web Experience</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="noise"></div>

  <header class="nav">
    <a class="brand" href="#">NOVA<span>.</span></a>
    <nav>
      <a href="#features">Features</a>
      <a href="#about">About</a>
      <a class="nav-btn" href="#contact">Launch</a>
    </nav>
  </header>

  <main>
    <section class="hero">
      <div class="eyebrow">SELF-HOSTED • FAST • PRIVATE</div>
      <h1>Your website.<br><em>Your server.</em></h1>
      <p class="hero-copy">
        A polished starter site designed to run beautifully from a tiny
        Android server powered by Termux.
      </p>
      <div class="actions">
        <a class="btn primary" href="#features">Explore</a>
        <a class="btn ghost" href="#about">Learn more</a>
      </div>
      <div class="orb orb-a"></div>
      <div class="orb orb-b"></div>
    </section>

    <section id="features" class="section">
      <div class="section-head">
        <span>01</span>
        <h2>Built for the web.</h2>
      </div>
      <div class="grid">
        <article class="card">
          <span class="icon">⬆</span>
          <h3>Responsive</h3>
          <p>Designed for phones first, then scaled smoothly to desktop.</p>
        </article>
        <article class="card">
          <span class="icon">⚡</span>
          <h3>Lightweight</h3>
          <p>Plain HTML, CSS and JavaScript. No framework required.</p>
        </article>
        <article class="card">
          <span class="icon">∞</span>
          <h3>Self-hosted</h3>
          <p>Your files live on your own device and are served by Caddy.</p>
        </article>
      </div>
    </section>

    <section id="about" class="split">
      <div>
        <span class="eyebrow">02 / THE IDEA</span>
        <h2>Small hardware.<br>Big presence.</h2>
      </div>
```

```

    <p>
      Replace this copy with your story, portfolio, shop, business,
      documentation or project. This template intentionally has no
      external dependencies, so it is easy to host from Termux.
    </p>
  </section>

  <section id="contact" class="cta">
    <p class="eyebrow">03 / READY</p>
    <h2>Make the Internet<br>your canvas.</h2>
    <a class="btn primary" href="mailto:hello@example.com">Contact</a>
  </section>
</main>

<footer>
  <span>NOVA.</span>
  <span>Hosted from Termux</span>
</footer>

<script src="script.js"></script>
</body>
</html>

```

style.css:

```

*{box-sizing:border-box;margin:0;padding:0}
:root{--bg:#07070a;--text:#f4f4f5;--muted:#a1a1aa;--line:#24242b;--accent:#b8ff6a}
html{scroll-behavior:smooth}
body{font-family:Inter,ui-sans-serif,system-ui,-apple-system,Segoe UI,sans-serif;background:var(--bg);color:var(--text);line-height:1.6;overflow:
a{color:inherit;text-decoration:none}
.noise{position:fixed;inset:0;pointer-events:none;opacity:.035;background-image:url("data:image/svg+xml,%3Csvg viewBox='0 0 180 180' xmlns='http:
.nav{height:76px;display:flex;align-items:center;justify-content:space-between;padding:0 6vw;border-bottom:1px solid var(--line);position:relativ
.brand{font-weight:900;letter-spacing:.16em}.brand span{color:var(--accent)}}
nav{display:flex;align-items:center;gap:28px;font-size:.9rem;color:var(--muted)}
nav a: hover{color:var(--text)}.nav-btn{border:1px solid var(--line);padding:9px 16px;border-radius:999px}
.hero{min-height:calc(100vh - 76px);display:flex;flex-direction:column;justify-content:center;padding:10vh 9vw;position:relative;isolation:isolat
.eyebrow{font-size:.72rem;letter-spacing:.22em;color:var(--muted);font-weight:800}
.hero h1{font-size:clamp(4rem,12vw,9rem);line-height:1.86;letter-spacing:-.065em;max-width:1000px;margin:24px 0}
.hero h1 em{font-style:normal;color:var(--accent)}
.hero-copy{max-width:580px;color:var(--muted);font-size:clamp(1rem,2vw,1.25rem)}
.actions{display:flex;gap:12px;margin-top:32px}.btn{display:inline-flex;padding:13px 20px;border-radius:999px;font-weight:800;border:1px solid va
.primary{background:var(--accent);color:#0a0a0a;border-color:var(--accent)}
.ghost{color:var(--text)}
.orb{position:absolute;border-radius:50%;filter:blur(3px);z-index:-1}
.orb-a{width:380px;height:380px;right:4vw;top:15%;background:radial-gradient(circle,#b8ff6a44,transparent 65%)}
.orb-b{width:500px;height:500px;left:-15%;bottom:-25%;background:radial-gradient(circle,#8b5cf633,transparent 65%)}
.section,.split,.cta{padding:110px 9vw;border-top:1px solid var(--line)}
.section-head{display:flex;gap:30px;align-items:baseline;margin-bottom:40px}.section-head span{color:var(--accent);font-family:monospace}.section
.grid{display:grid;grid-template-columns:repeat(3,1fr);gap:14px}.card{border:1px solid var(--line);padding:30px;min-height:220px;border-radius:24
.split{display:grid;grid-template-columns:1fr 1fr;gap:8vw;align-items:end}.split>p{font-size:1.15rem;max-width:560px}
.cta{text-align:center}.cta h2{margin:25px auto 35px}.cta .btn{display:inline-flex}
footer{display:flex;justify-content:space-between;padding:30px 9vw;color:var(--muted);font-size:.8rem;border-top:1px solid var(--line)}
@media(max-width:760px){nav a:not(.nav-btn){display:none}.hero{padding-left:7vw;padding-right:7vw}.grid,.split{grid-template-columns:1fr}.section

```

script.js:

```

const reveal = new IntersectionObserver((entries)=>{
  entries.forEach(e=>{
    if(e.isIntersecting){
      e.target.animate(
        [{opacity:0, transform:"translateY(22px)"},
          {opacity:1, transform:"translateY(0)"}],
        {duration:650, easing:"cubic-bezier(.2,.7,.2,1)", fill:"forwards"}
      );
      reveal.unobserve(e.target);
    }
  });
},{threshold:.12});

document.querySelectorAll(".card,.split,.cta").forEach(el=>reveal.observe(el));

```

To serve this example, change the Caddy root to the premium-site directory, validate, then run
~ /start-site.

11. Recommended final file structure

```
$HOME/
├── .duckdns_token          # secret; chmod 600
├── start-site              # one-command launcher
├── duckdns/
│   └── update-ipv6.sh      # optional standalone updater
├── caddy/
│   └── Caddyfile
├── testserver/
│   └── YOUR_WEBSITE_DIRECTORY/
│       ├── caddy           # custom Caddy + DuckDNS module
│       ├── index.html
│       ├── style.css
│       └── script.js
└── ~/.termux/
    ├── boot/
    │   └── start-site      # reboot launcher
```

Keep credentials outside your website directory. Never upload `.duckdns_token`, a token-containing script, Caddy private storage, or certificate/private-key files to GitHub.

12. Troubleshooting — common errors and exact fixes

caddy: bind: permission denied on :443

Normal for an unrooted Termux process. Use HTTPS on 8443 and DNS-01. Do not try to solve this by randomly changing permissions.

caddy: bind: permission denied on :80

Caddy's HTTP-01 challenge and automatic redirect normally use port 80. Disable automatic redirects and use DNS-01: `auto_https disable_redirects + tls { dns duckdns {env.DUCKDNS_API_TOKEN} }`.

could not start listener for challenge server at :80

This is the same privileged-port problem. DNS-01 removes the need for port 80 during certificate issuance.

dns.providers.duckdns is missing

The installed Caddy binary lacks the DuckDNS module. Build with: `xcaddy build --with github.com/caddy-dns/duckdns`. Verify with `caddy list-modules | grep dns.providers.duckdns`.

android/arm requires external (cgo) linking, but cgo is not enabled

Install clang, set `CGO_ENABLED=1`, then rerun `xcaddy build`.

DuckDNS returns KO

Check the token, hostname, and network connection. Never print the token in public logs. Run the updater manually and inspect only the final result.

No usable global IPv6

Check `ip -6 route show default` and `ip -6 addr`. Toggle mobile data or airplane mode, then retry. The script uses the source IPv6 selected for an external route.

Website works on the phone but not from outside

Test from a different network. Confirm the AAAA record and externally test TCP 8080/8443 over IPv6. Also confirm the carrier permits inbound IPv6.

The domain works for some people but not others

You may be IPv6-only. Clients without IPv6 cannot directly reach the server. Use a tunnel or dual-stack service if you need IPv4 reachability.

The website disappears after reboot

Install/open Termux:Boot once, put the executable boot script in `~/.termux/boot/`, and remove battery optimization restrictions for Termux where possible.

Caddy certificate renewal fails later

Ensure DuckDNS token remains valid, DNS API access works, and Caddy's storage directory is persistent. DNS-01 itself does not require port 80/443.

13. Security and reliability checklist

- Keep Android, Termux and Caddy updated.
- Serve only files intended to be public. Never expose your home directory or secret files.
- Keep the DuckDNS token in a chmod 600 file outside the website directory.
- Do not commit tokens, Caddy storage, private keys, or personal data to Git repositories.
- Use HTTPS for public sites; do not send credentials over plain HTTP.
- Do not run unnecessary network services on the phone.
- Use a strong Android lock screen and device encryption.
- Expect mobile IPv6 addresses to change. The DDNS updater is therefore part of the hosting system.
- Use a charger and monitor heat/battery health for long-running operation.
- A phone is not equivalent to a redundant production server: reboots, carrier changes, battery restrictions, radio changes and Android process management can cause downtime.

Privacy note

A public server necessarily exposes an Internet endpoint and will receive unsolicited scanning and HTTP requests. Caddy access logs and the web server itself may reveal request metadata. Don't put private information in public web content.

14. Why the URL has :8443

The standard HTTPS URL is `https://example.com`, which implies TCP 443. Unrooted Android applications normally cannot bind to privileged ports below 1024. Therefore this guide uses 8443.

The result is:

```
https://YOURNAME.duckdns.org:8443
```

DNS-01 certificate issuance does not care that your HTTPS service is on 8443; the ACME validation happens through DNS TXT records.

A clean port-443 URL generally requires one of: root-level port redirection, a reverse proxy/tunnel, or another network component that can receive 443 and forward it to 8443. If you use a tunnel, the Android remains the origin server but traffic is no longer a purely direct mobile-IPv6 connection.

15. Quick deployment checklist for other people

- 1) Install Termux
- 2) Get mobile IPv6 working
- 3) Create DuckDNS hostname
- 4) Save token:
 `~/.duckdns_token (chmod 600)`
- 5) Put website files in:
 `~/testserver/YOUR_WEBSITE_DIRECTORY/`
- 6) Install:
 `golang git clang`
- 7) Install xcaddy
- 8) Build:
 `xcaddy build --with github.com/caddy-dns/duckdns`
- 9) Verify:
 `./caddy list-modules | grep dns.providers.duckdns`
- 10) Create Caddyfile with:
 `https_port 8443
 auto_https disable_redirects
 tls { dns duckdns {env.DUCKDNS_API_TOKEN} }`
- 11) Create `~/start-site`
- 12) Run:
 `~/start-site`
- 13) Test:
 `https://YOURNAME.duckdns.org:8443`
- 14) Install Termux:Boot
- 15) Put boot launcher in:
 `~/.termux/boot/start-site`

This checklist is intentionally based on the working architecture in this guide, including the two problems that commonly appear on unrooted Termux: privileged ports and a missing DuckDNS Caddy module.

16. Official / primary references

Caddy Automatic HTTPS — challenge behavior and DNS-01:

<https://caddyserver.com/docs/automatic-https>

Caddy TLS directive — DNS challenge configuration:

<https://caddyserver.com/docs/caddyfile/directives/tls>

DuckDNS Caddy DNS provider module:

<https://github.com/caddy-dns/duckdns>

DuckDNS:

<https://www.duckdns.org/>

Termux:Boot:

<https://github.com/termux/termux-boot>

Caddy custom builds / xcaddy:

<https://caddyserver.com/docs/build>

Final note: This is a practical self-hosting guide, not a guarantee that every mobile carrier/device will provide inbound IPv6. Carrier firewalling, IPv6 availability, Android power management, and address changes vary. Always test from an external IPv6 network before calling a deployment production-ready.

17. Final copy-paste cheat sheet

After the one-time setup is complete, the everyday command is simply:

```
~/start-site
```

The intended public address is:

```
https://YOURNAME.duckdns.org:8443
```

To stop it:

```
CTRL+C
```

To update DNS manually:

```
~/duckdns/update-ipv6.sh
```

To inspect the Caddy configuration:

```
./caddy validate --config ~/caddy/Caddyfile
```

To confirm the DuckDNS module exists:

```
./caddy list-modules | grep dns.providers.duckdns
```

Congratulations — you now have a reproducible blueprint for hosting a static website directly from an Android/Termux server with free DNS, automated IPv6 updates, and HTTPS.